You can leave out the `:on` option, this will create the same member route except that the resource id value will be available in `params[:photo_id]` instead of `params[:id]`.

2.10.2 Adding Collection Routes

To add a route to the collection:

```
resources :photos do
  collection do
    get 'search'
  end
end
```

This will enable Rails to recognize paths such as `/photos/search` with GET, and route to the `search` action of `PhotosController`. It will also create the `search_photos_url` and `search_photos_path` route helpers.

Just as with member routes, you can pass `:on` to a route:

```
resources :photos do
  get 'search', on: :collection
end
```

2.10.3 Adding Routes for Additional New Actions

To add an alternate new action using the `:on` shortcut:

```
resources :comments do
  get 'preview', on: :new
end
```

This will enable Rails to recognize paths such as `/comments/new/preview` with GET, and route to the `preview` action of `CommentsController`. It will also create the `preview_new_comment_url` and `preview_new_comment_path` route helpers.

If you find yourself adding many extra actions to a resourceful route, it's time to stop and ask yourself whether you're disguising the presence of another resource.

3 Non-Resourceful Routes

In addition to resource routing, Rails has powerful support for routing arbitrary URLs to actions. Here, you don't get groups of routes automatically generated by resourceful routing. Instead, you set up each route within your application separately.

While you should usually use resourceful routing, there are still many places where the simpler routing is more appropriate. There's no need to try to shoehorn every last piece of your application into a resourceful framework if that's not a good fit.

In particular, simple routing makes it very easy to map legacy URLs to new Rails actions.

3.1 Bound Parameters

When you set up a regular route, you supply a series of symbols that Rails maps to parts of an incoming HTTP request. Two of these symbols are special: `:controller` maps to the name of a controller in your application, and `:action` maps to the name of an action within that controller. For example, consider this route:

```
get ':controller(/:action(/:id))'
```

If an incoming request of `/photos/show/1` is processed by this route (because it hasn't matched any previous route in the file), then the result will be to invoke the `show` action of the `PhotosController`, and to make the final parameter `"1"` available as `params[:id]`. This route will also route the incoming request